
LEARNING TO WALK RANDOMLY

Anonymous author

ABSTRACT

This report investigates the task of bipedal locomotion using the rldurham/Walker environment under both the easy mode (`hardcore=False`) and the challenging mode (`hardcore=True`). The TD3 [2] algorithm is primarily employed to train the agent, leveraging experience replay and delayed policy updates to optimize continuous action strategies. Through this approach, the agent successfully learns stable walking behaviors even in complex and uneven terrains.

1 METHODOLOGY

1.1 ENVIRONMENT AND ACTION SPACE

This study is based on the rldurham/Walker environment, which is built upon the classic BipedalWalker environment from OpenAI Gym, aiming to train a bipedal walker robot to achieve stable locomotion over complex and uneven terrains. The environment provides the agent with a continuous observation space and action space. The action space consists of four control dimensions, corresponding to the torque or velocity commands applied to the hip and knee joints on both the left and right sides:

$$a = [a_{\text{hip1}}, a_{\text{knee1}}, a_{\text{hip2}}, a_{\text{knee2}}]$$

Each action dimension is bounded within the range of $[-1, +1]$, ensuring the control signals remain within the physical limits. The details of the action space are summarized below:

	Joint Name	Minimum	Maximum
0	Left Hip (Hip 1)	-1	+1
1	Left Knee (Knee 1)	-1	+1
2	Right Hip (Hip 2)	-1	+1
3	Right Knee (Knee 2)	-1	+1

Table 1: Definition of the action space. The actor network uses a tanh activation function at the output layer to ensure that the generated actions comply with the physical constraints.

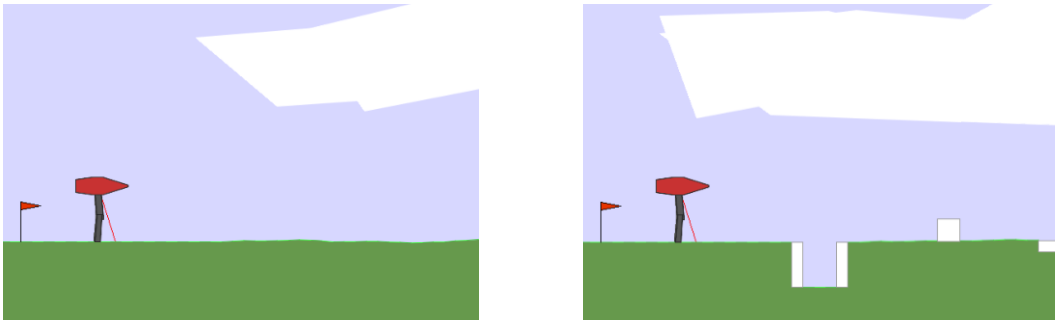


Figure 1: This figure illustrates the initial terrain configurations of the Walker environment under the easy mode (`hardcore=False`) and the hard mode (`hardcore=True`).

1.2 ACTOR AND CRITIC NETWORKS

The TD3-based [2] agent training in this study is built upon the Actor-Critic framework, where the actor network is responsible for generating continuous actions and the critic networks are responsible for evaluating the value of actions. Specifically, TD3 introduces twin critic networks to reduce overestimation bias.

Actor Network: The actor network takes the current state as input and passes it through multiple fully connected layers with hidden sizes of 512, 256, 128, and 64, respectively. Each hidden layer uses the ReLU activation function, and the final output layer applies a tanh activation function to ensure that the action values are bounded within an appropriate range [3]. It is worth noting that the actor network includes one more hidden layer than the critic network, enhancing its capacity to model the mapping from the state space to the action space.

Critic Network: The critic network consists of two independent Q-functions, Q_1 and Q_2 . Each Q-network takes the concatenated state-action vector as input and passes it through three hidden layers with sizes of 512, 256, and 128, respectively, to output a single scalar Q-value. Each hidden layer uses the ReLU activation function. By employing twin Q-networks and using the $\min(Q_1, Q_2)$ strategy, TD3 effectively mitigates the problem of Q-value overestimation [2]. Compared to the actor network, the critic has a slightly shallower architecture with fewer hidden layers to reduce the risk of overfitting the value function. As shown in the figure 2 below:

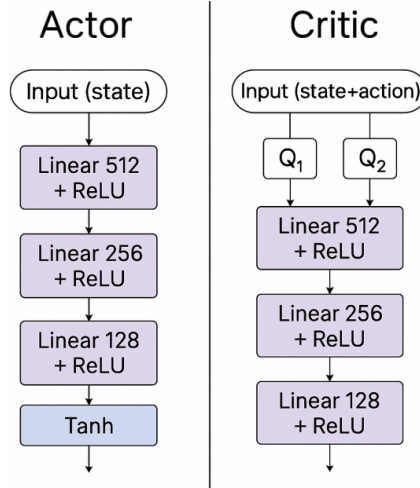


Figure 2: Architectural comparison between the Actor and Critic networks in our TD3 implementation.

1.3 TRAINING PROCEDURE

To achieve a more effective and stable learning process, this study incorporates techniques such as Experience Replay and Soft Target Updates during training to improve agent performance and final outcomes.

Experience Replay [4]: A replay buffer is used to store transition tuples (s, a, r, s', d) collected during the agent’s interaction with the environment. During training, small batches of samples are randomly drawn from the buffer to update the networks, which helps to break the correlation between consecutive samples and significantly improves training stability.

Soft Target Updates [2]: Separate target networks are maintained for both the actor and critic. These target networks are updated using a soft update strategy to slowly track the parameters of the main networks. The update rules are given by:

$$\theta_{\text{target}} \leftarrow \tau \theta + (1 - \tau) \theta_{\text{target}}$$

$$\phi_{\text{target}} \leftarrow \tau \phi + (1 - \tau) \phi_{\text{target}}$$

where θ and ϕ represent the parameters of the current actor and critic networks, respectively, and $\tau = 0.01$ is the soft update rate. This technique effectively reduces the estimation oscillations and improves training stability.

Critic Target (TD Target) [2]: In TD3, the critic networks are updated using a target value that incorporates target policy smoothing. The target value is defined as:

$$y = r + \gamma(1 - d) \min(Q_1(s', \tilde{a}), Q_2(s', \tilde{a}))$$

where r is the immediate reward, d is the done signal (1 if terminated, 0 otherwise), and $\tilde{a} = \pi_{\text{target}}(s') + \epsilon$ is the next action with clipped Gaussian noise $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma^2), -c, c)$. By taking the minimum of the two critic estimates, TD3 mitigates overestimation bias [2].

Delayed Policy Updates [2]: In TD3, the actor network is updated less frequently than the critic networks. Specifically, the actor is updated once every two critic updates, ensuring that the policy is optimized with respect to sufficiently accurate value estimates. The actor’s optimization objective is to maximize the Q-value of the current state-action pair:

$$\max_{\theta} \mathbb{E}_{s \sim D}[Q(s, \pi(s|\theta))]$$

Exploration Noise [2]: To encourage exploration during training, Gaussian noise with mean 0 and standard deviation 0.2 is added to the actions output by the actor network during environment interaction.

Target Policy Smoothing Noise [2]: When computing the TD target, small clipped Gaussian noise is added to the target action $\pi_{\text{target}}(s')$ to smooth the value estimation and prevent the policy from exploiting sharp peaks in the value function.

Warm-up Phase [2]: At the beginning of training, the agent interacts with the environment using purely random actions to enhance data diversity and exploration coverage. Specifically, in the easy mode (`hardcore=False`), random actions are used for the first 20,000 steps, while in the hard mode (`hardcore=True`), random actions are used for the first 50,000 steps. After the warm-up phase, the agent switches to policy-based action selection for targeted optimization.

2 CONVERGENCE RESULTS

To evaluate the training performance of the TD3 algorithm under different settings, this study compares the agent’s learning behavior in the Walker environment under both the easy mode (`hardcore=False`) and the hard mode (`hardcore=True`). In addition to differing in the duration of random actions at the beginning of training, the hard mode also increases the maximum number of steps per episode from 2000 (in easy mode) to 4000 to accommodate the more complex terrain and allow for more thorough exploration. The training curves show that while the agent can quickly converge and achieve stable walking behavior in the easy mode, it faces significant challenges in the hard mode, with reduced training stability and overall performance.

2.1 `HARDCORE=False`

In the easy mode (`hardcore=False`), the agent demonstrates good convergence performance. As shown in Fig. 3, the agent begins to significantly improve its score around the 600th episode and stabilizes after approximately 1000 episodes. The final average return remains around 300 with relatively low variance, indicating that the policy has reliably learned to accomplish the task.

2.2 `HARDCORE=True`

In contrast, in the hard mode (`hardcore=True`), the agent’s learning process is significantly more challenging. As shown in Fig. 4, the agent starts to show some improvement after

approximately 1500 episodes. However, the overall training process exhibits considerable fluctuations, with some final scores exceeding 290 while others stabilize around 100. The convergence is much slower, and the final average return remains relatively low, fluctuating around 100. This suggests that more training time and more efficient policy optimization are required for the agent to achieve stable locomotion in complex terrains.

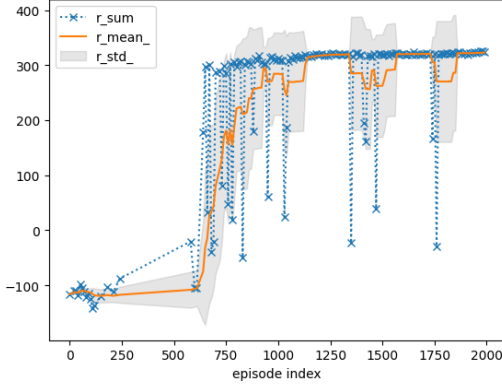


Figure 3: Training rewards in the easy mode (hardcore=False).

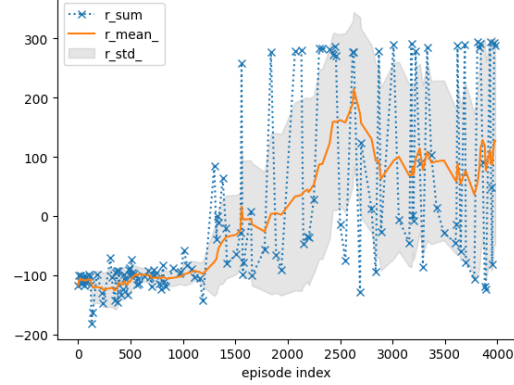


Figure 4: Training rewards in the hard mode (hardcore=True).

3 LIMITATIONS

Although the TD3-based agent demonstrates good convergence performance in the easy mode (hardcore=False), there are still several limitations in the overall training process. In the hard mode (hardcore=True), the agent’s learning becomes significantly more difficult. Even after several thousand training episodes, only limited performance improvement is observed, with the average return fluctuating around 100 and exhibiting high variance, indicating a slower convergence rate.

Moreover, during the early and even mid-training stages, the agent often fails to produce effective walking behavior, resulting in zero or even negative rewards—effectively showing a “failure to walk.” This phenomenon suggests that the current policy architecture and training mechanism struggle to achieve stable generalization in complex terrains, especially in scenarios with sparse rewards and abrupt terrain changes.

FUTURE WORK

Finally, experimental results show that in the easy mode, the agent is able to achieve good convergence within 2000 episodes, with the final score stably maintained above 300. However, in the hard mode, due to the increased terrain complexity, the agent’s learning process becomes significantly more challenging, characterized by slower convergence and larger performance fluctuations. The best episode scores can reach around 290, but in some cases, the scores may even be negative, and the final average score stabilizes around 100. These results indicate that while TD3 can effectively solve moderately difficult locomotion tasks, it still faces considerable challenges in complex environments.

To address these issues, future work can explore several directions to further enhance the agent’s walking performance over complex terrains. First, extending the training time and adopting larger batch sizes may help improve the stability of policy convergence [1]. Second, exploring more advanced reinforcement learning algorithms such as DrQ-v2 [6] could improve sample efficiency and generalization ability. In addition, hierarchical reinforcement learning methods [5] can be employed to decompose tasks into manageable subgoals, thereby better handling complex environments.

REFERENCES

- [1] Gabriel Barth-Maroon et al. *Distributed Distributional Deterministic Policy Gradients*. 2018. arXiv: 1804.08617 [cs.LG]. URL: <https://arxiv.org/abs/1804.08617>.
- [2] Scott Fujimoto, Herke van Hoof, and David Meger. *Addressing Function Approximation Error in Actor-Critic Methods*. 2018. arXiv: 1802.09477 [cs.AI]. URL: <https://arxiv.org/abs/1802.09477>.
- [3] Timothy P. Lillicrap et al. *Continuous control with deep reinforcement learning*. 2019. arXiv: 1509.02971 [cs.LG]. URL: <https://arxiv.org/abs/1509.02971>.
- [4] Long-Ji Lin. “Self-improving reactive agents based on reinforcement learning, planning and teaching”. In: *Machine Learning* 8.3 (1992), pp. 293–321. DOI: 10.1007/BF00992699. URL: <https://doi.org/10.1007/BF00992699>.
- [5] Alexander Sasha Vezhnevets et al. *FeUdal Networks for Hierarchical Reinforcement Learning*. 2017. arXiv: 1703.01161 [cs.AI]. URL: <https://arxiv.org/abs/1703.01161>.
- [6] Denis Yarats et al. *Mastering Visual Continuous Control: Improved Data-Augmented Reinforcement Learning*. 2021. arXiv: 2107.09645 [cs.AI]. URL: <https://arxiv.org/abs/2107.09645>.